



Security Audit Report

Stride: LSM support

Authors: Darko Deuric, Marko Juric, Nikola Jovicevic

Last revised 1 June, 2023

Table of Contents

Audit Overview	1
The Project	1
Scope of this audit	1
Conducted work	1
Timeline	1
Conclusions	2
Audit Dashboard	3
Target Summary	3
Engagement Summary	3
Severity Summary	3
System Overview.....	4
LSM Liquid staking	4
Unbonding: Motivation and Algorithm	5
Threat Inspection	6
LSM liquid stake flow related threats	6
Unbonding related threats	8
Findings	10
Not all ICAs are considered to prevent shared resource changes	11
Failure of IBC Transfer or ICA Detokenization, but stTokens Already Issued to the User	13
A validator could be mistakenly considered as slashed even if they have not actually been slashed	15
If the total unbond amount is less than the total LSM portion of stake, the unbonding amount can seep into the native portion of stake for a given validator	18
The ending delegation for a validator can be different than if algorithm did a rebalancing after the unbonding	20
User can get stTokens but the LSMTokenDeposit could be removed before detokenization	22
LSM liquid staking might be locked for a specific validator	24
Appendix: Vulnerability Classification	26
Impact Score	26
Exploitability Score	26
Severity Score	27

Disclaimer..... 29

Audit Overview

The Project

In May 2023, the Stride development team engaged Informal Systems to conduct a security audit of the Liquidity Staking Module (LSM) support on Stride. The LSM module, implemented by Iqlusion, is planned to be added to the Cosmos Hub through a [signaling proposal](#). This module allows delegators to tokenize their stakes without waiting for the unbonding period, providing them with LSM tokens. These LSM tokens can be sent to Stride, where users can receive stTokens.

Scope of this audit

The audit was conducted by the following individuals:

- Darko Deuric
- Marko Juric
- Nikola Jovicevic

During the audit, our primary focus was on analyzing the modifications made in the x/stakeibc module, specifically the implementation of LSM liquid stake support and the new unbonding algorithm. The Stride team had specific expectations from us, which included paying special attention to and ensuring the correctness of the slashing query case modeling (which is a part of entry flow to the LSM). Additionally, we were tasked with examining edge cases related to error handling, timeouts, IBC transfers, ICAs, and ICQs.

Conducted work

As part of the audit project, the team conducted the following work:

1. Review of Specification and Technical Diagrams:
 - The team thoroughly reviewed the [specification](#) and technical diagrams ([d1](#) and [d2](#)) related to the LSM liquid stake flow and unbonding algorithm.
2. Manual Code Review:
 - A comprehensive line-by-line manual code review was performed on all parts of the codebase that were modified to support LSM. The primary focus of this review was to ensure the correctness of the LSM liquid stake flow and the unbonding algorithm.
3. Formal Verification Models using Quint:
 - The team created two formal verification models using Quint:
 - ICA/ICQ Overlap: The model was designed to analyze and verify the handling of overlap scenarios between ICAs and ICQs.
 - Unbonding Algorithm: The model was developed to verify the correctness and integrity of the unbonding algorithm implemented in the system.

Timeline

Week 1:

- Kickoff meeting with the Stride team, where we discussed the LSM Liquid stake use case. We received the current available documentation from them.

Week 2:

- Discussed the first potential issues related to the LSM liquid stake flow. The Stride team provided us with additional materials, including diagrams and videos shared by Sam. They emphasized the importance of examining the slashing queries to the host in the LSM liquid stake flow and the unbonding algorithm.

Week 3:

- Presented the threat model to the Stride team, specifically focusing on the LSM liquid stake flow. We discussed the unbonding algorithm and identified the invariants that needed to be checked using the quint model.

Week 4:

- Provided the Stride team with the results of the quint verification for the unbonding algorithm. Additionally, they requested us to perform a quint specification for the ICA/ICQ overlap.

Week 5:

- Conclusions and final findings from the Stride audit were presented and shared with the Stride team.

Conclusions

We are highly impressed with the code changes implemented in the x/stakeibc module, particularly regarding the LSM liquid stake support. It is evident that the Stride team has thoroughly considered every aspect of the LSM liquid staking flow and the unbonding case. The attention to detail is commendable, especially in handling sensitive components such as:

- IBC/ICA/ICQ responses,
- slashing ICQ,
- ICA/ICQ overlaps,
- unbonding algorithm, and
- the creation of comprehensive diagrams that provide both a high-level overview and relevant details.

In terms of code quality, we are pleased to report that we did not come across any critical, high, or even medium severity issues in the codebase. Most of the issues we encountered were related to scenarios that the Stride team was already aware of, further demonstrating the high quality of the codebase.

Furthermore, we have witnessed the potential of utilizing the Quint modelling language for writing code specifications, particularly in two cases: ICA/ICQ overlap and the unbonding algorithm. For instance, Quint uncovered a [variation in ICA/ICQ calls](#) that would have been challenging to identify through manual analysis alone. This experience encourages us to explore Quint further in future audits if the need arises.

Overall, we are thoroughly impressed with the well-thought-out code changes, the attention to detail, and the high quality of the codebase. Your efforts have been exemplary, and we look forward to further discussions and collaborations in the future.

Audit Dashboard

Target Summary

- **Type:** Specification and Implementation
- **Platform:** Golang
- **Artifacts:**
 - Commit hash: a41faa32668fa5ccd39378901de756fa3db79e93
 - [Repo](#)

Engagement Summary

- **Dates:** 03.05.2023 to 26.05.2023
- **Method:** Manual code review, protocol analysis, formal modeling
- **Employees Engaged:** 3

Severity Summary

Finding Severity	#
Critical	0
High	0
Medium	0
Low	4
Informational	3
Total	7

System Overview

LSM Liquid staking

To provide an introduction, let's highlight the main reasons **why Stride aimed to integrate LSM**:

1. Improving the Liquid Staking User Experience:
 - Currently, if you hold staked atoms, you need to unbond them before liquid staking, resulting in a loss of 3 weeks of interest.
 - By implementing LSM support, Stride aims to enhance the user experience of liquid staking.
2. Establishing a Safety Framework for the Cosmos Hub:
 - LSM introduces regulations for Liquid Staking (LS) providers.
 - These regulations include a *GlobalLiquidStakingCap*, restricting LS providers to hold a maximum of 25% of the total staked Atoms.
 - Additionally, the *ValidatorBondFactor* helps address the [principal-agent problem](#) by incentivizing validators to self-bond their tokens.
3. Encouraging Validators to Self-Bond:
 - Presently, only a few validators have self-bonds.
 - Liquid staking provides an incentive for validators to self-bond their tokens, thereby enhancing the overall security and stability of the Cosmos Hub.

Here is a **typical user flow**:

1. Users delegate their tokens or already have tokens staked.
2. They initiate a `MsgTokenizeShares` transaction, which rewards them with LSM tokens (NFT), representing their shares or the right to take over the delegation later.
3. The LSM token is transferred via IBC to Stride, triggering the `LSMLiquidStake` process.
4. The user receives stTokens in return.

Behind the scenes, when the LSM token is sent to Stride, it is forwarded to Delegation ICA on Host Zone. On the callback, an ICA is issued, which detokenizes the LSM tokens by converting them back into native delegation using the `MsgRedeemTokensForShares` transaction.

Let's delve deeper into the **LSM liquid stake flow on Stride**:

Prerequisite: The user has transferred LSM tokens from the Host Zone to Stride.

1. User triggers the `LSMLiquidStake` message.
2. The message is validated, ensuring the correctness of the channel ID, denom, validator address, and balance.
3. If the slash query checkpoint (determined as a percentage of Total Value Locked) has been reached, the system checks if the validator has been slashed (Step 4). If the checkpoint has not been reached, the process proceeds to Step 6.
4. A validator slash query is submitted (with `ValidatorExchangeRateCallback`). If the validator exchange rate has not changed, the process proceeds to Step 6.
5. If the validator exchange rate has changed, the system calls `DelegationsIcq` to calculate the magnitude of the slash and update the relevant data (such as `validator.Delegation`, `validator.Weight`, `hostZone.TotalDelegations`, `hostZone.Validators`) accordingly, specifically in the `DelegatorSharesCallback`.
6. The LSM liquid stake process is completed (`FinishLSMLiquidStake`). The LSM token is sent to the Deposit account, and stTokens are minted and sent to the user.

There was no need to visualize the flow as the Stride team has already created a comprehensive and suitable [diagram](#) for it.

Unbonding: Motivation and Algorithm

Motivation:

When LSM liquid stakes are received, they cannot be evenly distributed among Stride's different validator weights. Normally, liquid stakes are delegated evenly based on weights, but LSM liquid stakes are specific to a validator.

In the SDK, there is a constraint that prevents rebalancing more than once per unbonding period. Over the course of three weeks (for Cosmos Hub), during this unbonding period, the distribution of stake becomes increasingly imbalanced.

Therefore, the approach taken is to unbond whenever there is an opportunity, gradually moving the stakes towards a more balanced state.

Algorithm steps:

1. Determine the total amount to be unbonded.
2. Calculate the ideal distribution of stake after the unbonding process.
3. Determine each validator's capacity, which is defined as the difference between their current stake distribution and the ideal distribution.
4. Calculate the balance ratio for each validator, which determines the priority for unbonding.
5. Iterate through the validators in order of priority and unbond as much as possible from each validator, up to their capacity, until the total amount has been unbonded.

Once again, the Stride developers have provided an excellent [diagram](#) illustrating the unbonding process, including various scenarios and cases.

Threat Inspection

The threats presented here are not necessarily a direct result of inspecting the code but rather a product of considering various attack vectors after reading the specification and examining the diagrams as well as formal modeling and protocol verification. Some of the threats were identified as findings (✗), while others were determined to be harmless (✓).

LSM liquid stake flow related threats

- **Threat 1: Locking of LSM Liquid Stakings for a Specific Validator** ✗
 - The `SlashQueryInProgress` field has been added to the validator struct to prevent any new LSM liquid stakes from being accepted while a slash query is in progress. It is of utmost importance to handle the `SlashQueryInProgress` field appropriately and address all possible edge cases. This is necessary to avoid potential LSM liquid staking locks on specific validators.
 - <https://informalsystems.atlassian.net/l/cp/sQ79Kswm> arises from this threat
- **Threat 2: Delayed Update of Redemption Rate after Slashing** ✓
 - The assumption is that the user could obtain stTokens at an unfair price if the slash query result is deemed "positive." This scenario could occur if the redemption rate is not promptly updated following the update of all relevant data on Stride's side, such as the validator's delegation and weight, as well as the host zone's total delegation and validators. The actual code flow for this situation is as follows:
 - When a validator is slashed in the `DelegatorSharesCallback`, the appropriate state is updated, including(`validator.Weight`, `validator.Delegation` and `hostZone.Validators`)
 - Additionally, there is an `UpdateRedemptionRateForHostZone` process that updates the redemption rate accordingly.
 - Previously, the redemption rate was updated only during the `STRIDE_EPOCH` process. However, an improvement has been made by adding the `UpdateRedemptionRateForHostZone` function at the end of the `SlashValidatorOnHostZone` function. This ensures that the redemption rate is now promptly updated immediately after a slash is detected. Therefore, there is no need to worry about delays in updating the redemption rate.
 - This indicates that the threat is harmless, and the user will receive stTokens based on the updated redemption rate.
- **Threat 3: Sending stTokens to the User if Validator Was Slashed** ✓
 - When user initiates LSM liquid staking, it is important to ensure that the user won't get stTokens in the case of validator slashing detection
 - To avoid progressing towards `FinishLSMLiquidStake` (where actual minting and sending stTokens occurs) if a validator was slashed, the implementation ensures that `FinishLSMLiquidStake` can only be called from two possible points:
 1. `ValidatorExchangeRateCallback`, if there was no slash (async call)
 2. `LSMLiquidStake` if there were no callbacks at all (sync call)
 This flow guarantees the safe handling of stTokens in such cases.
- **Threat 4: Race Conditions and Shared State Variables (validator.Delegation)** ✗
 - It is important to consider the possibility of overlap between ICA and ICQ, as described in detail in Stride's [specs](#).
 - Considering the two possible cases of interest: ICA ack landing before ICQ Ack, or vice versa, it is important to check whether any action modifies shared resources, such as `validator.Delegation`, while another action is in progress.

- `Delegation ICA`, `Undelegation ICA` and `Detokenization ICA` have been checked against the status of the corresponding record (Is it `IN_PROGRESS`). However, there is one more callback, `RebalanceCallback`, that modifies `validator.Delegation` but is not handled in the `CheckDelegationChangedDuringQuery` function.
- Not all ICAs are considered to prevent shared resource changes arises from this threat
- **Threat 5: There is a possibility that stTokens may be issued to the user while the processing of the corresponding real LSM token by Stride may not be fully completed** ❌
 - In the unlikely event of a failed IBC ack, the user may be left with stTokens, but the corresponding real LSM tokens might not have been transferred or detokenized.
 - The LSM token becomes "stuck" on Stride or Host.
 - `Failure of IBC Transfer or ICA Detokenization, but stTokens Already Issued to the User` arises from this threat

Threats found using quint ICA/ICQ overlap model:

ICA/ICQ overlap according to the Stride team have two possible permutations (snippet from code):

```

Permutation #1
    ICA Lands on Host                                ICQ Ack on Stride
    ICA Lands on Host    ICA Ack on Stride    ICQ Lands on Host
Permutation #2
    ICA Lands on Host                                ICA Ack on Stride
    ICA Lands on Host    ICQ Lands on Host    ICQ Ack on Stride
  
```

Invariant to be tested:

Regardless of the ordering of different actions, the delegation in our record-keeping should never be updated to a value that is out of sync

Threats 6 and 7 outlined below describe situations where there is a possibility of updating the delegation in Stride's record-keeping, potentially leading to it being out of sync.

- **Threat 6: A validator could be mistakenly considered as slashed even if they have not actually been slashed** ❌
 - In the event of ICA/ICQ overlap during the LSM liquid stake flow, when an ICQ acknowledges, the expected behavior is as follows: It checks
 - if the delegation has changed since the initiation of the query or
 - if there are any delegation changes in progress. If either condition is met, the query is retried.
 - However, if the ICAs complete before the ICQ acknowledgment and leave the delegation amount unchanged (e.g., +5, -5), and if there are no changes in progress since all ICAs have finished, it creates an opportunity for the delegation to be updated. In such a scenario, the ICQ acknowledgment could potentially send an "older" value since the ICAs occurred in the meantime.
 - `A validator could be mistakenly considered as slashed even if they have not actually been slashed` arises from this threat
- **Threat 7: A validator could be mistakenly considered as NOT slashed even if they have been slashed.**



- We can also consider another potential scenario, which we can refer to as permutation #3

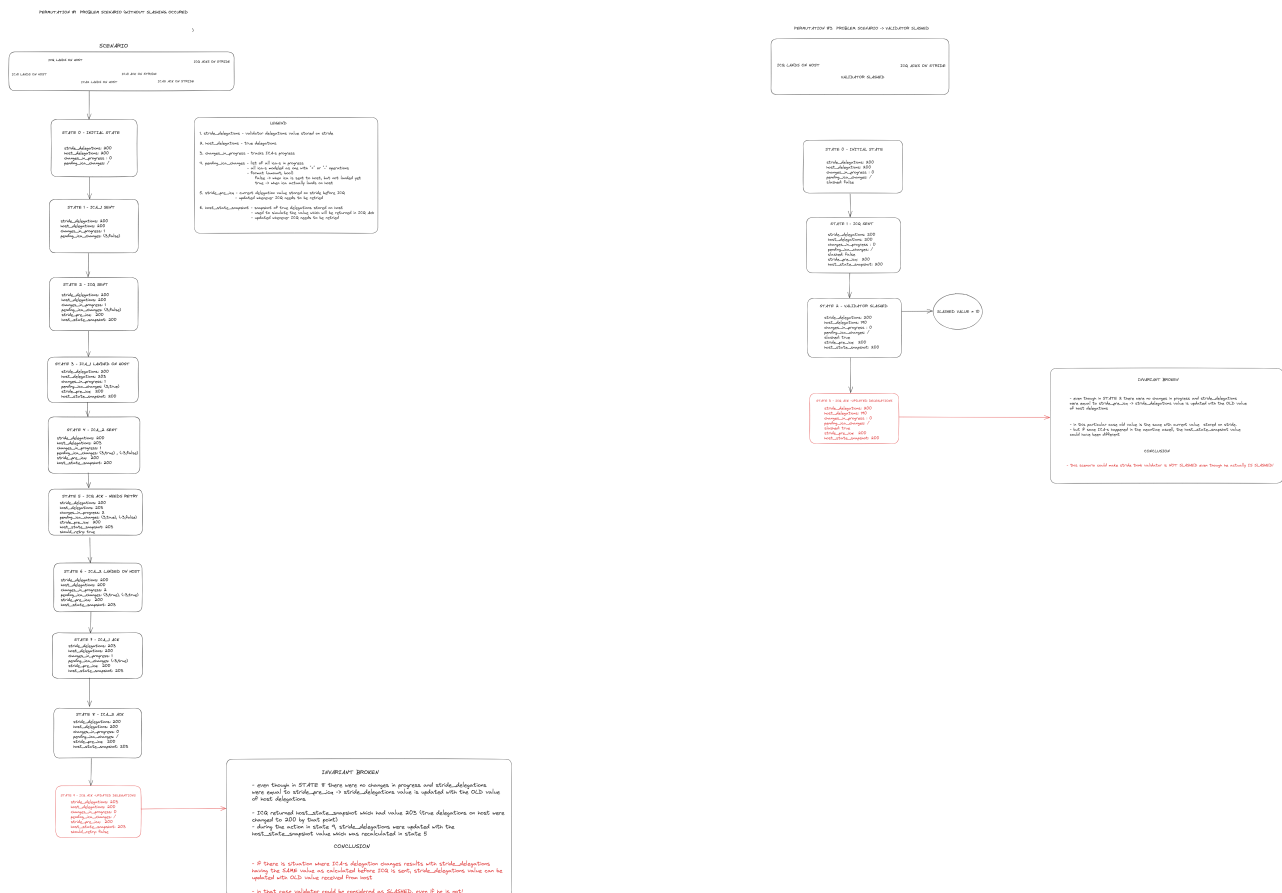
```

Permutation #3
    ICA Lands on Host    Validator slashed    ICA Ack on Stride    ICQ Lands on Host    ICQ Ack on Stride
  
```

- ICA landing on the Host and ICA acknowledgment on Stride are not necessary for this scenario. What is crucial is that the validator slashing occurs between the ICQ landing on the Host and the ICQ acknowledgment.
- In this situation, the delegation amount may remain the same as it was in Stride prior to the slashing event. Furthermore, there would be no ICAs in progress, leaving the opportunity for delegations to be

updated. The ICQ acknowledgment could potentially send an old value, reflecting the state before the slashing incident took place.

- The Stride team has determined that this situation is acceptable because the query is only issued once a change in the exchange rate is detected. Additionally, it implies that the validator was not slashed when the ICQ was initiated, thereby nullifying the attack vector as it indicates that the liquid stake occurred prior to any slashing event.



Scenario 6 and 7 examples described on diagram

Unbonding related threats

Invariants to be tested:

1. **Unbonding amount should never be zero or negative** ✓
2. **We should never attempt to unbond more than the validator has delegated to it** ✓
3. **Sum of total stake after unbonding and unbonding amount should always be equal to initial total stake** ✓
4. **Sum of unbond amounts from ICA messages should equal total unbond amount** ✓
5. **If the total unbond amount is less than the total LSM portion of stake, the unbonding amount should never seep into the native portion of stake for a given validator** ✗

- The maximum amount of natives taken off that resulted in breaking the invariant was 10.
- The unbonding algorithm on Stride does not differentiate between LSM and Native Stake, so it is mathematically possible to "break this invariant" in practice.
- However, it was expected that this invariant would fail, at least due to the potential precision leak when performing operations with integers.

- We were unable to find examples that break the invariant with larger differences, indicating that the algorithm does not allow such scenarios. However, further investigation can be conducted if necessary.
- Following discussions with the Stride team, they advised us to proceed with the ICQ/ICA overlap model instead of delving further into this particular scenario.
- <https://informalsystems.atlassian.net/l/cp/WJuo3ztC> arises from this threat

6. If the unbond amount is less than the total delegations to zero weight validators, there should only be unbondings from zero weight validators 

7. The ending delegation for a validator should be no lower than if we did a rebalancing after the unbonding



One of the results observed:

- validator_total_stakes: Map("A" -> 6110, "B" -> 9349, "C" -> 7955, "D" -> 2535)
- rebalanced_stakes_after_unbonding: Map("A" -> 6110, "B" -> 9341, "C" -> 7955, "D" -> 2543)

Once again, there appear to be differences in the values due to imprecision. However, we did not conduct further investigation into this matter as we proceeded with the ICQ/ICA overlap model instead.

<https://informalsystems.atlassian.net/l/cp/1ZA7tbfx> arises from this threat

Findings

Title	Type	Severity	Impact	Exploitability	Issue
If the total unbond amount is less than the total LSM portion of stake, the unbonding amount can seep into the native portion of stake for a given validator	IMPLEMENTATION	0 INFORMATIONAL	0 NONE	0 NONE	
The ending delegation for a validator can be different than if algorithm did a rebalancing after the unbonding	IMPLEMENTATION	0 INFORMATIONAL	0 NONE	0 NONE	
LSM liquid staking might be locked for a specific validator	IMPLEMENTATION	0 INFORMATIONAL	2 MEDIUM	0 NONE	
Not all ICAs are considered to prevent shared resource changes	IMPLEMENTATION	1 LOW	1 LOW	1 LOW	https://github.com/Stride-Labs/stride/pull/772/files
Failure of IBC Transfer or ICA Detokenization, but stTokens Already Issued to the User	IMPLEMENTATION	1 LOW	2 MEDIUM	1 LOW	
A validator could be mistakenly considered as slashed even if they have not actually been slashed	IMPLEMENTATION	1 LOW	1 LOW	0 NONE	
User can get stTokens but the LSMTOKENDeposit could be removed before detokenization	IMPLEMENTATION	1 LOW	2 MEDIUM	1 LOW	https://github.com/Stride-Labs/stride/pull/783

Not all ICAs are considered to prevent shared resource changes

Title	Not all ICAs are considered to prevent shared resource changes
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	1 LOW
Impact	1 LOW
Exploitability	1 LOW
Issue	https://github.com/Stride-Labs/stride/pull/772/files

Involved artifacts

- [x/stakeibc/keeper/icqcallbacks_delegator_shares.go](https://github.com/Stride-Labs/stride/blob/main/x/stakeibc/keeper/icqcallbacks_delegator_shares.go)

Description

Due to the asynchronous behavior of ICAs and ICQs, it is crucial to prevent race conditions between an ICA that changes delegation for a specific validator and the query response for that validator's total delegation.

In order to address this, it is important to consider two possible scenarios: either the ICA acknowledgment arrives before the ICQ acknowledgment or vice versa. In either case, it is essential to check whether any action is modifying shared resources, such as the `validator.Delegation`, while another action is already in progress. This ensures that no conflicts or unintended modifications occur during the process.

Problem Scenarios

The status of `Delegation ICA`, `Undelegation ICA` and `Detokenization ICA` has been verified against the corresponding record's status, specifically checking if it is `IN_PROGRESS`. However, there is another callback, `RebalanceCallback`, which modifies `validator.Delegation` but is currently not accounted for in the `CheckDelegationChangedDuringQuery` function:

```
...
// Check that there are no deposit records in state IN_PROGRESS - indicative of a
// Delegation ICA
for _, depositRecord := range k.RecordsKeeper.GetAllDepositRecord(ctx) {
    if depositRecord.HostZoneId == chainId &&
        depositRecord.Status == recordstypes.DepositRecord_DELEGATION_IN_PROGRESS
    {
        k.Logger(ctx).Error("Delegation ICA is currently in progress. Rejecting
        query callback and resubmitting query")
    }
}
```

```

        return true, nil
    }
}

// Check that there are no epoch unbonding records in state IN_PROGRESS -
// indicative of an Undelegation ICA
// TODO: This is expensive, we should store these more efficiently
for _, epochUnbondingRecord := range
k.RecordsKeeper.GetAllEpochUnbondingRecord(ctx) {
    for _, hostUnbondingRecord := range epochUnbondingRecord.HostZoneUnbondings {
        if hostUnbondingRecord.HostZoneId == chainId &&
            hostUnbondingRecord.Status ==
recordstypes.HostZoneUnbonding_UNBONDING_IN_PROGRESS {
            k.Logger(ctx).Error("Undelegation ICA is currently in progress.
Rejecting query callback and resubmitting query")
            return true, nil
        }
    }
}

// Check that there are no LSMTOKENDEPOSITS in state IN_PROGRESS - indictive of
an Detokenization ICA
detokenizationInProgress :=
recordstypes.LSMTOKENDEPOSIT_DETOKENIZATION_IN_PROGRESS
detokenizationInProgressRecords :=
k.RecordsKeeper.GetLSMTokenDepositsForHostZoneWithStatus(ctx, chainId,
detokenizationInProgress)
for _, lsmTokenDeposit := range detokenizationInProgressRecords {
    if lsmTokenDeposit.ValidatorAddress == validatorAddress {
        k.Logger(ctx).Error("Detokenization ICA is currently in progress.
Rejecting query callback and resubmitting query")
        return true, nil
    }
}
...

```

The impact of this situation is considered **Low** because the potential consequence is that `validator.Delegation` could be slightly out of sync for particular validator. Furthermore, the exploitability of this issue is considered **Low**, as it is unlikely for this scenario to occur in real-life circumstances.

Recommendation

To address this issue, it is recommended to handle the `RebalanceCallback` within the `CheckDelegationChangedDuringQuery` function. Alternatively, you can explore ways to create a general approach that supports any additional callbacks that may modify shared resources.

Status Update

Resolved in [PR](#). This PR adds a field to the validator that increments every time a relevant ICA is submitted, and decrements when in the callback for that ICA. Then, the delegator shares query callback can simply check that there are no ICAs in progress using this field, rather than looping across all the records. This field also helps outline this assumption more explicitly.

Failure of IBC Transfer or ICA Detokenization, but stTokens Already Issued to the User

Title	Failure of IBC Transfer or ICA Detokenization, but stTokens Already Issued to the User
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	1 LOW
Impact	2 MEDIUM
Exploitability	1 LOW
Issue	

Involved artifacts

- [x/stakeibc/keeper/icacallbacks_detokenize.go](#)
- [x/records/keeper/callback_lsm_transfer.go](#)

Description

It is crucial to ensure that both of the following events occur:

1. The user receives stTokens after providing the LSM token.
2. The LSM token is successfully transferred to the Host Zone, followed by the detokenization process.

For Step 2 to be considered successful, both the acknowledgments corresponding to the IBC transfer and the ICA detokenization must be successful.

Problem Scenarios

In the unlikely event of a failed IBC acknowledgment (exploitability **Low**), the user may end up with stTokens, while the corresponding real LSM tokens might not have been transferred or detokenized (impact **Medium**).

If an IBC ack fails, LSM tokens can become "stuck" on Stride, as there is no retry mechanism in place for failed IBC transfers. Since the `LSMTokenDeposit` is not removed in this scenario, the user cannot perform LSM liquid staking with the same denomination if they attempted to liquid stake partially in the first attempt.

Regarding detokenization, if a timeout occurs, the channel (as it is an ICA related) will be closed and cannot be retried. To handle this, a separate transaction is required to restore the channel and update the status back to `LSMTokenDeposit_DETOKENIZATION_QUEUE`, allowing it to be resubmitted. However, in the case of an ack failure, the deposit will be marked with `LSMTokenDeposit_DETOKENIZATION_FAILED` status, and similar to the IBC transfer ack failure scenario, LSM tokens can become "stuck" on Host Zone.

When could an ack fail?

The most likely case is when the relayer is at fault for failed IBC acks. For example, the relayer might submit packets without corresponding client updates or packets with incorrect proofs or sequence numbers. This would result in the chain rejecting the packets. The receiving chain can also have misconfigured Keepers for example or incorrect app/module plumbing.

In the less likely case, a failed ack could occur due to incorrect packet payloads containing invalid denominations or invalid destination accounts, among other possibilities.

Recommendation

It is important to carefully consider all possible cases where an acknowledgement (ack) could fail, especially when the relayer is at fault. It is recommended to thoroughly analyze and plan for these scenarios, taking into account off-chain monitoring as a means of detecting and addressing any failed acks.

Status Update

The Stride team is fully aware of this particular scenario. They believe that the likelihood of an ack failing is extremely low. Additionally, they have implemented off-chain monitoring to detect and handle any instances of failed transfers.

A validator could be mistakenly considered as slashed even if they have not actually been slashed

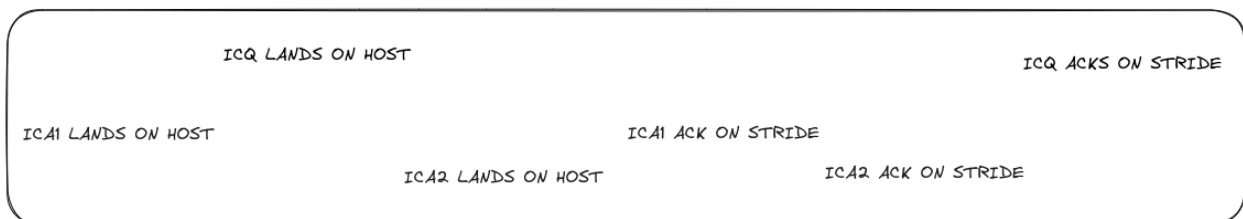
Title	A validator could be mistakenly considered as slashed even if they have not actually been slashed
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	1 LOW
Impact	1 LOW
Exploitability	0 NONE
Issue	

Involved artifacts

- x/stakeibc/keeper/icqcallbacks_delegator_shares.go

Description

SCENARIO



When Stride wants to check if a validator has been slashed, it sends an ICQ to the host. Upon receiving the ICQ Ack (acknowledgement), Stride needs to ensure that the delegation field in the ICQ Ack matches the true delegation amount on the host at the time the ICQ landed. However, there is a potential scenario where an ICA lands before the ICQ and completes between the ICQ landing and the ICQ Ack. Meanwhile, another ICA lands and receives an Ack during this timeframe. The ICA can take various forms, such as Delegation ICA, Unbonding ICA, Rebalance ICA, or Detokenization ICA.

After sending the ICA, the `delegation_changes_in_progress` field is incremented to indicate that a delegation change process is currently underway for the validator. When the acknowledgement (Ack) for the ICA is received, the `delegation_changes_in_progress` field is decremented, reflecting the completion of the action.

In case of an overlap between ICA and ICQ during the LSM liquid stake flow, if Stride wants to verify whether a validator has been slashed before proceeding further, the expected behavior upon receiving the ICQ Ack is as follows:

- When the ICQ Ack is received, Stride checks if the delegation in it's record keeping has changed from the time the query was initially initiated
- Additionally, Stride checks if the number of `delegation_changes_in_progress` is non-zero. This field reflects the ongoing delegation changes for the validator.
- if either condition is met (delegation has changed or `delegation_changes_in_progress` is non-zero), Stride retries the query to ensure an accurate and up-to-date response.

This is how Stride wants to avoid the scenario where regardless of the ordering of those different actions, stride never updates the delegation in it's record keeping to a value that's out of sync.

Problem Scenarios

In some cases, even with checks in place, there is still a possibility of updating the delegations with an out-of-sync value. One such scenario is when all ICAs finish before the ICQ Ack is received and leave the delegation amount the same as before, resulting in a potential inconsistency. Let's consider the following example:

- Initial delegation value in Stride's record-keeping: 200.
- True delegation value: 200.
- `delegation_changes_in_progress` = 0

Here's the sequence of events:

1. ICA 1 lands on the host, changing the true delegation to 150, and `delegation_changes_in_progress` is incremented to 1.
2. ICQ lands on the host, reflecting the true delegation value of 150.
3. ICA 2 lands on the host, changing the true delegation back to 200, and `delegation_changes_in_progress` is incremented to 2.
4. ICA 1 Ack is received by Stride, updating the value in Stride's record-keeping to 150, and `delegation_changes_in_progress` is decremented to 1.
5. ICA 2 Ack is received by Stride, updating the value in Stride's record-keeping to 200, and `delegation_changes_in_progress` is decremented to 0.
6. ICQ Ack is received by Stride, attempting to update Stride's record-keeping with the value 150.

In summary, despite `delegation_changes_in_progress` being 0 and the delegation value in Stride's record-keeping being unchanged before the ICQ is sent, Stride is still updated with the older value of 150 instead of the true delegation value of 200 at that moment. This creates a discrepancy and highlights the potential for inconsistencies in the process.

Recommendation

On the one hand this scenario is not very likely to happen often since there needs to be a combination of parameters matched in order to meet conditions, but on the other hand it opens possibility to treat some user like his validator is slashed, which may not be true.

A possibility to avoid this risk is to have some kind of counter which will count ICAs in progress, but ones which are sent in between ICQ sent and ICQ Ack. When ICQ Ack arrives, it could check whether any ICA is sent after the ICQ sent, and if that's true, ICQ can be resent.

While the occurrence of the described scenario may be unlikely due to the specific conditions that need to be met, it still poses a risk of incorrectly treating a user as if their validator has been slashed when it may not be the case.

To mitigate this risk, one possible approach is to implement a counter that tracks the number of ICAs in progress specifically sent between the time the ICQ is sent and when the ICQ Ack is received. Here's a cleaner explanation:

1. When an ICQ is sent, a counter starts to track the number of ICAs that are sent after the ICQ is initiated but before the ICQ Ack is received.
2. When the ICQ Ack is received, it checks whether any ICAs have been sent after the ICQ was initiated.
3. If the check reveals that ICAs were indeed sent during that period, it indicates the possibility of overlapping actions.
4. In such a case, the ICQ can be resent to ensure an accurate and up-to-date response, thereby avoiding any potential inconsistencies caused by overlapping ICAs.

Status Update

Stride team is aware of the situation and are opened to recommendations of solving the potential risk.

If the total unbond amount is less than the total LSM portion of stake, the unbonding amount can seep into the native portion of stake for a given validator

Title	If the total unbond amount is less than the total LSM portion of stake, the unbonding amount can seep into the native portion of stake for a given validator
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	0 NONE
Issue	

Involved artifacts

- x/stakeibc/keeper/unbonding_records.go

Description

The unbonding algorithm in the provided diagram illustrates four possible cases that can occur during the unbonding process. These cases are:

1. Unbonding less than the total amount of zero weight delegations.
2. Unbonding less than the total LSM stake amount.
3. Unbonding the entire LSM stake amount.
4. Unbonding more than the total LSM stake amount.

It is important to note that when examining the examples in the diagram, it appears that native stakes are not touched unless all LSM stakes are unbonded first. This suggests that the unbonding algorithm steps are designed to cover this scenario as well.

Problem Scenarios

In the provided scenario, the implementation of the unbonding algorithm does not differentiate between native and LSM stakes. It calculates ideal distributions and validator capacities based on the validator weights and then sorts the validators according to a ratio to determine the unbonding order.

Consider the following information for the four validators (A, B, C, and D):

- validator weights: A = 64, B = 41, C = 80, D = 67
- native stakes: A = 7536, B = 4826, C = 9421, D = 7900

- total native stakes: 29683
- lsm stakes: A = 5149, B = 2340, C = 7528, D = 3961
- total lsm stakes: 8978
- total stakes: 48661
- total unbond amount: 18976

After the unbonding process, the following results are observed:

- ideal distributions: A = 7537, B = 4826, C = 9432, D = 7890
- validator capacities: A = 5148, B = 2340, C = 7517, D = 3971
- unbonding messages: C = 7517, A = 5148, D = 3971, B = 2340
- native stakes: A = 7536, B = 4826, C = 9421, D = 7890
- lsm stakes: A = 1, B = 0, C = 11, D = 0

It is evident that validator D had 3961 of LSM stakes, but the algorithm unbonded 3971 from them. This resulted in the algorithm having to utilize a portion of the native stake of validator D to unbond the remaining 10.

On the other hand, validator C still had 11 LSM stake remaining. In theory, it would have been possible to unbond an additional 10 from the LSM stake of validator C instead of utilizing the native portion of validator D.

Recommendation

The current implementation of the algorithm aims to achieve a balanced state, and any changes to prevent the scenarios described above would likely increase the complexity of the algorithm. The existing algorithm, although relatively simple, works towards a more balanced distribution.

It is important to note that no examples were found where a significant amount of stakes was taken from the native portion, exceeding a relatively small number (in the context of distributions).

How we tested with quint is that all native stakes were distributed to the validators according to the weights. Then, LSM stakes were non-deterministically distributed to all validators. There were also constants used for defining ranges from which non deterministic function could pick a value.

During the testing process using the quint specification, the distribution of native stakes to validators was determined based on their weights. On the other hand, LSM stakes were distributed in a non-deterministic manner among all validators. Crucially, ranges were defined to limit the maximum amount of stakes that could be distributed.

It's worth emphasizing that the size of the range directly affects the potential difference that could lead to a violation of the desired invariant. In simpler terms, as the maximum value defined within the range increases, the potential difference that could arise also increases. In the specific example provided, the defined ranges ensured that the difference observed did not exceed a value of 10.

Status Update

Stride team is aware of this scenario, in particular of the fact that algorithm implementation doesn't recognize difference between native and LSM stakes which opens door for scenarios like this.

The ending delegation for a validator can be different than if algorithm did a rebalancing after the unbonding

Title	The ending delegation for a validator can be different than if algorithm did a rebalancing after the unbonding
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	0 NONE
Issue	

Involved artifacts

- [x/stakeibc/keeper/unbonding_records.go](https://x.stakeibc/keeper/unbonding_records.go)

Description

As part of the testing process for the Unbonding algorithm, one of the defined invariants was that the ending delegation for a validator should not be lower than if a rebalancing were performed after the unbonding process.

To test this invariant using the "quint" framework, the following approach was taken: after the Unbonding algorithm completed, the remaining stakes of each validator were considered. An ideal distribution was then calculated based on these remaining stakes, taking into account the validator weights. The expectation was that the resulting distribution would be the same as the input value of the remaining validator stakes.

Problem Scenarios

The defined invariant, which states that the ending delegation for a validator should not be lower than if a rebalancing were performed after the unbonding process, has been observed to fail in the majority of cases. However, it is important to note that the differences observed are typically quite small.

For example, consider the following result:

- remaining validator stakes: A: 6110, B: 9349, C: 7955, D: 2535
- rebalanced stakes after unbonding: A: 6110, B: 9341, C: 7955, D: 2543

In this case, the rebalanced stakes after the unbonding process differ slightly from the original remaining validator stakes. Although the differences are small, they still violate the defined invariant.

Recommendation

The observed differences in stake distributions between the original remaining validator stakes and the rebalanced stakes after unbonding can likely be attributed to mathematical imprecision with integers. Further investigation into this scenario was not pursued at the time, as due to the instructions from Stride team, the focus shifted to the ICQ/ICA overlap model.

However, it is crucial to highlight the important invariants that have been validated and demonstrate the correctness of the algorithm:

- Unbonding amount should never be zero or negative
- We should never attempt to unbond more than the validator has delegated to it
- Sum of unbond amounts from ICA messages should equal total unbond amount
- If the unbond amount is less than the total delegations to zero weight validators, there should only be unbondings from zero weight validators

Status Update

Stride team is informed that this invariant is violated.

User can get stTokens but the LSMTokenDeposit could be removed before detokenization

Title	User can get stTokens but the LSMTokenDeposit could be removed before detokenization
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	1 LOW
Impact	2 MEDIUM
Exploitability	1 LOW
Issue	https://github.com/Stride-Labs/stride/pull/783

Involved artifacts

- x/stakeibc/keeper/icqcallbacks_validator_exchange_rate.go

Description

In the code version that was audited, there was an implementation of validator exchange rate callback permutations, which can be observed in the LSM Liquid Stake [diagram](#). One of the cases specifically addressed the situation where the validator was not slashed, and the callback was triggered from the LSM liquid stake flow. In this scenario, the flow should conclude by calling `FinishLSMLiquidStake` and issuing stTokens to the user.

Furthermore, it is evident from the provided code snippet that if `FinishLSMLiquidStake` fails, the `RemoveLSMTokenDeposit` function will be invoked.

```

    if !validatorWasSlashed && inLSMLiquidStakeCallback {
        k.Logger(ctx).Info(utils.LogICQCallbackWithHostZone(chainId,
            ICQCallbackID_Validator,
            "Validator was not slashed, finishing LSM liquid stake"))

        async := true
        if err := k.FinishLSMLiquidStake(ctx, lsmLiquidStake, async); err != nil {
            // If finishing the transaction failed, emit an event, but swallow the
            error so
            // the query object still gets deleted
            errorMessage := fmt.Sprintf("lsm liquid stake callback failed after slash
            query: %s", err.Error())
            EmitFailedLSMLiquidStakeEvent(ctx, hostZone, lsmTokenDeposit,
            errorMessage)
            k.Logger(ctx).Error(errorMessage)
        }
    }

```

```
        // Remove the LSMTokenDeposit
        k.RecordsKeeper.RemoveLSMTokenDeposit(ctx,
lsmliquidstake.Deposit.ChainId, lsmliquidstake.Deposit.Denom)
    }
    return nil
}
```

Problem Scenarios

Inside the `FinishLSMLiquidStake` function, there is a `flow` that has the potential to fail. This failure can lead to an inconvenient scenario where `FinishLSMLiquidStake` is partially completed, the user receives `stTokens`, but the `LSMTokenDeposit` is deleted.

As a result, there is a risk that Stride might never detokenize the user's LSM token, and the user could potentially unbind someone else's native assets, resulting in a **Medium** impact. However, it is important to note that the likelihood of such a scenario occurring is relatively low, such as due to marshaling errors, host zone or validator not being found, or errors returned by `AfterLiquidStake`. In terms of exploitability, this issue is considered **Low**.

Recommendation

Sam recommended calling `FinishLSMLiquidStake` with a temporary context that can be discarded in case of failure. This approach is highly recommended to handle the situation of `FinishLSMLiquidStake` being partially completed.

Status Update

Resolved in [PR](#). We can call `FinishLSMLiquidStake` with a temporary context that gets discarded in the event of an error - reverting state changes in the process. This way, we can swallow the error (allowing the Query object to get deleted), while discarding any of the partial state changes that may have occurred at the start of `FinishLSMLiquidStake`.

LSM liquid staking might be locked for a specific validator

Title	LSM liquid staking might be locked for a specific validator
Project	Stride: LSM support
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	2 MEDIUM
Exploitability	0 NONE
Issue	

Involved artifacts

- x/interchainquery/keeper/msg_server.go

Description

At the beginning of the `ValidatorExchangeRateCallback`, the variable `validator.SlashQueryInProgress` is set to `true`, indicating that a slash query is in progress.

The `LSMSlashQueryTimeout` is set to 5 minutes, which could potentially lead to a timeout scenario in the `DelegatorSharesCallback`.

In the event of a timeout, the EndBlocker of the Interchainquery module includes a `RetryICQRequest`. This retry eventually proceeds with the second callback, sets `validator.SlashQueryInProgress` to `false`, and releases the validator for other LSM liquid stakings.

Problem Scenarios

In the event of a query response error, such as an `unverified query proof`, the `SubmitQueryResponse` function will return an error. Consequently, `validator.SlashQueryInProgress` will remain `true`, rendering the validator unavailable for subsequent LSM operations. The impact of this issue is considered **Medium**, as it primarily affects a single validator but has the potential to impact a significant number of users who have tokenized shares from that validator.

However, the exploitability of this scenario is deemed **None** due to the extremely low likelihood of query proof verification failure.

Recommendation

It would be beneficial to consider the possibility of reverting `validator.SlashQueryInProgress` in the event of a query proof failure within the `SubmitQueryResponse` function. However, due to limited input parameters, it is unlikely to be feasible. As an alternative, implementing off-chain monitoring to track events such as query failures would be a suitable approach to cover this scenario effectively.

Status Update

Stride team is aware of the situation and they will off-chain monitor events like query failures.

Appendix: Vulnerability Classification

For classifying vulnerabilities identified in the findings of this report, we employ the simplified version of [Common Vulnerability Scoring System \(CVSS\) v3.1](#), which is an industry standard vulnerability metric. For each identified vulnerability we assess the scores from the *Base Metric Group*, the [Impact score](#), and the [Exploitability score](#). The *Exploitability score* reflects the ease and technical means by which the vulnerability can be exploited. That is, it represents characteristics of the *thing that is vulnerable*, which we refer to formally as the *vulnerable component*. The *Impact score* reflects the direct consequence of a successful exploit, and represents the consequence to the *thing that suffers the impact*, which we refer to formally as the *impacted component*. In order to ease score understanding, we employ [CVSS Qualitative Severity Rating Scale](#), and abstract numerical scores into the textual representation; we construct the final *Severity score* based on the combination of the Impact and Exploitability sub-scores.

As blockchains are a fast evolving field, we evaluate the scores not only for the present state of the system, but also for the state that deems achievable within 1 year of projected system evolution. E.g., if at present the system interacts with 1-2 other blockchains, but plans to expand interaction to 10-20 within the next year, we evaluate the impact, exploitability, and severity scores wrt. the latter state, in order to give the system designers better understanding of the vulnerabilities that need to be addressed in the near future.

Impact Score

The Impact score captures the effects of a successfully exploited vulnerability on the component that suffers the worst outcome that is most directly and predictably associated with the attack.

Impact Score	Examples
High	Halting of the chain; loss, locking, or unauthorized withdrawal of funds of many users; arbitrary transaction execution; forging of user messages / circumvention of authorization logic
Medium	Temporary denial of service / substantial unexpected delays in processing user requests (e.g. many hours/days); loss, locking, or unauthorized withdrawal of funds of a single user / few users; failures during transaction execution (e.g. out of gas errors); substantial increase in node computational requirements (e.g. 10x)
Low	Transient unexpected delays in processing user requests (e.g. minutes/a few hours); Medium increase in node computational requirements (e.g. 2x); any kind of problem that affects end users, but can be repaired by manual intervention (e.g. a special transaction)
 None	Small increase in node computational requirements (e.g. 20%); code inefficiencies; bad code practices; lack/incompleteness of tests; lack/incompleteness of documentation

Exploitability Score

The Exploitability score reflects the ease and technical means by which the vulnerability can be exploited; it represents the characteristics of the vulnerable component. In the below table we list, for each category, examples of actions by actors that are enough to trigger the exploit. In the examples below:

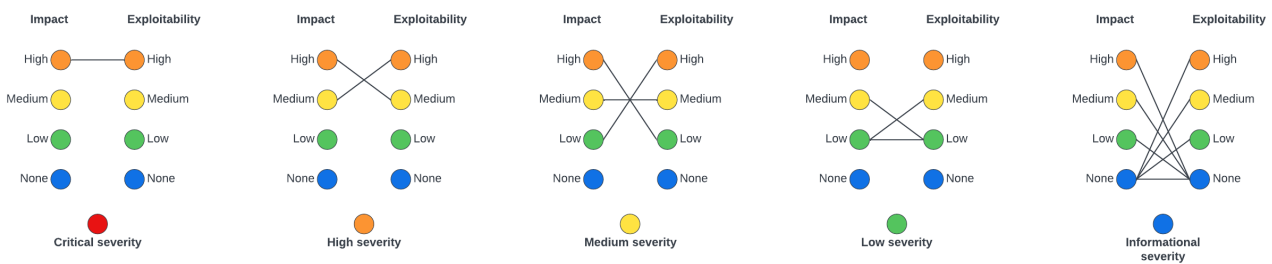
- *Actors* can be any entity that interacts with the system: other blockchains, system users, validators, relayers, but also uncontrollable phenomena (e.g. network delays or partitions).
- *Actions* can be

- *legitimate*, e.g. submission of a transaction that follows protocol rules by a user; delegation/redelegation/bonding/unbonding; validator downtime; validator voting on a single, but alternative block; delays in relaying certain messages, or speeding up relaying other messages;
- *illegitimate*, e.g. submission of a specially crafted transaction (not following the protocol, or e.g. with large/incorrect values); voting on two different alternative blocks; alteration of relayed messages.
- We employ also a *qualitative measure* representing the amount of certain class of power (e.g. possessed tokens, validator power, relayed messages): *small* for < 3%; *medium* for 3-10%; *large* for 10-33%, *all* for >33%. We further quantify this qualitative measure as relative to the largest of the system components. (e.g. when two blockchains are interacting, one with a large capitalization, and another with a small capitalization, we employ *small* wrt. the number of tokens held, if it is small wrt. the large blockchain, even if it is large wrt. the small blockchain)

Exploitability Score	Examples
High	illegitimate actions taken by a small group of actors; possibly coordinated with legitimate actions taken by a medium group of actors
Medium	illegitimate actions taken by a medium group of actors; possibly coordinated with legitimate actions taken by a large group of actors
Low	illegitimate actions taken by a large group of actors; possibly coordinated with legitimate actions taken by all actors
● None	illegitimate actions taken in a coordinated fashion by all actors

Severity Score

The severity score combines the above two sub-scores into a single value, and roughly represents the probability of the system suffering a severe impact with time; thus it also represents the measure of the urgency or order in which vulnerabilities need to be addressed. We assess the severity according to the combination scheme represented graphically below.



As can be seen from the image above, only a combination of high impact with high exploitability results in a Critical severity score; such vulnerabilities need to be addressed ASAP. Accordingly, High severity score receive vulnerabilities with the combination of high impact and medium exploitability, or medium impact, but high exploitability.

Severity Score	Examples
● Critical	Halting of chain via a submission of a specially crafted transaction

Severity Score	Examples
High	Permanent loss of user funds via a combination of submitting a specially crafted transaction with delaying of certain messages by a large portion of relayers
Medium	Substantial unexpected delays in processing user requests via a combination of delaying of certain messages by a large group of relayers with coordinated withdrawal of funds by a large group of users
Low	2x increase in node computational requirements via coordinated withdrawal of all user tokens
 Informational	Code inefficiencies; bad code practices; lack/incompleteness of tests; lack/incompleteness of documentation; any exploit for which a coordinated illegitimate action of all actors is necessary

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability, etc.) set forth in the associated Services Agreement. This report provided in connection with the Services set forth in the Services Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement.

This audit report is provided on an “as is” basis, with no guarantee of the completeness, accuracy, timeliness or of the results obtained by use of the information provided. Informal Systems has relied upon information and data provided by the client, and is not responsible for any errors or omissions in such information and data or results obtained from the use of that information or conclusions in this report. Informal Systems makes no warranty of any kind, express or implied, regarding the accuracy, adequacy, validity, reliability, availability or completeness of this report. This report should not be considered or utilized as a complete assessment of the overall utility, security or bugfree status of the code.

This audit report contains confidential information and is only intended for use by the client. Reuse or republication of the audit report other than as authorized by the client is prohibited.

This report is not, nor should it be considered, an “endorsement”, “approval” or “disapproval” of any particular project or team. This report is not, nor should it be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts with Informal Systems to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor does it provide any indication of the client’s business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should it be leveraged as investment advice of any sort.

Blockchain technology and cryptographic assets in general and by definition present a high level of ongoing risk. Client is responsible for its own due diligence and continuing security in this regard.